CS4265 Milestone 2 - Initial Implementation

April 23, 2026

1 What Works

In Milestone 2, I successfully built a working big data pipeline using Apache Spark and Amazon S3. The system can ingest data from multiple sources, store it in a persistent format, and read it back for verification. This shows that the project is feasible and ready for further development.

- **Multi-Source Data Acquisition**
  The pipeline successfully reads data from multiple heterogeneous sources, including Amazon Electronics reviews and metadata (JSON), Google Trends data (CSV), and a sample Common Crawl WET file (text). This demonstrates that the system can access and process different data formats.

```
===== Common Crawl WET Sample =====
root
 |-- value: string (nullable = true)

+----------------------------------------------------------------------+
|value                                                                 |
+----------------------------------------------------------------------+
|WARC/1.0                                                              |
|WARC-Type: warcinfo                                                   |
|WARC-Date: 2018-09-26T17:58:38Z                                       |
|WARC-Filename: CC-MAIN-20180918130631-20180918150631-00000.warc.wet.gz|
|WARC-Record-ID: <urn:uuid:a327852d-0a00-4a4d-9a92-61d9f9703f5a>       |
|Content-Type: application/warc-fields                                 |
|Content-Length: 374                                                   |
|                                                                      |
|Software-Info: ia-web-commons.1.1.9-SNAPSHOT-20180911015519           |
|Extracted-Date: Wed, 26 Sep 2018 17:58:38 GMT                         |
+----------------------------------------------------------------------+
only showing top 10 rows
Line count (sample action): 1000
```

Figure 1: common crawl data read 10 records

- **Schema-Based Processing**
  A reduced schema approach was used for the Amazon dataset due to inconsistent JSON structures. Only stable fields such as `asin`, `reviewText`, `overall`, and `unixReviewTime` were extracted to ensure reliable ingestion.

Figure 2: amazon review and metadata records

- **Distributed Processing with Spark**
  Apache Spark is used to perform distributed processing. Data is automatically split into partitions and processed in parallel. Even in local mode, the system demonstrates distributed execution behavior.

- **Parquet Output Pipeline**
  All datasets are written to Amazon S3 in Parquet format. This provides efficient columnar storage and improves performance for future analytics.

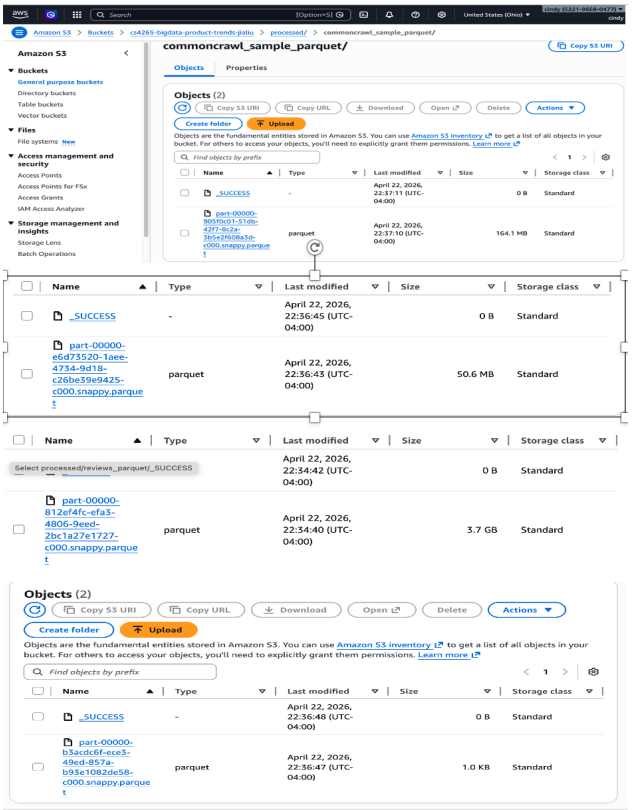

Figure 3: parquet

- **Read-Back Verification**
  After writing data to S3, the pipeline reads the Parquet files again to verify schema, sample records, and row counts. This confirms that the data is correctly stored and accessible.

```
only showing top 5 rows
Verification sample count: 1000

Verifying read-back for Google Trends from: s3a://cs4265-bigdata-product-trends-
jialiu/processed/trends_parquet/
root
 |-- Time: date (nullable = true)
 |-- electronics 2018: integer (nullable = true)

+----------+----------------+
|Time      |electronics 2018|
+----------+----------------+
|2017-12-31|13              |
|2018-01-07|22              |
|2018-01-14|11              |
|2018-01-21|11              |
|2018-01-28|9               |
+----------+----------------+
only showing top 5 rows
Verification sample count: 53

Verifying read-back for Common Crawl sample from: s3a://cs4265-bigdata-product-t
rends-jialiu/processed/commoncrawl_sample_parquet/
root
 |-- value: string (nullable = true)

+----------------------------------------------------------------------+
|value                                                                 |
+----------------------------------------------------------------------+
|WARC/1.0                                                              |
|WARC-Type: warcinfo                                                   |
|WARC-Date: 2018-09-26T17:58:38Z                                       |
|WARC-Filename: CC-MAIN-20180918130631-20180918150631-00000.warc.wet.gz|
|WARC-Record-ID: <urn:uuid:a327852d-0a00-4a4d-9a92-61d9f9703f5a>       |
+----------------------------------------------------------------------+
only showing top 5 rows
Verification sample count: 1000

M2 proof-of-concept pipeline completed successfully.
```

Figure 4: verificating the read-back

- **End-to-End Pipeline Execution**
  The system demonstrates a complete pipeline:
  - Read raw data from S3
  - Apply schema and filtering
  - Write processed data to S3
  - Read back output for validation

2 Big Data Technology Stack

The project is built on a layered Big Data technology stack integrating distributed storage, processing, and data formats.

Layer	Concept	Implementation
Data Source	Multi-source data	Amazon, Google Trends, Common Crawl
Storage	Distributed object storage	Amazon S3
Format	Data formats	JSON, CSV, WET, Parquet
Model	Structured data model	Spark DataFrame, Schema
Processing	Distributed computation	Apache Spark
Query/Output	Analytical-ready storage	Parquet + Spark read-back

This stack enables scalable ingestion, transformation, and storage of large datasets.

3 Pipeline Architecture

The data pipeline follows a distributed architecture integrating cloud storage and parallel processing:

- **Multi-Source Ingestion**
  Data is read from Amazon S3, including JSON, CSV, and WET files.
- **Data Processing**
  Spark applies schema enforcement and filtering to extract relevant fields while avoiding inconsistencies.
- **Transformation**
  A reduced schema is applied to simplify the dataset and ensure stable processing.
- **Storage**
  Processed data is written to S3 in Parquet format.
- **Validation**
  Data is read back from S3 to verify correctness and pipeline reliability.

This architecture demonstrates a scalable pipeline suitable for large-scale data processing.

4 Pipeline Diagram

```
Data Sources:
-------------------------------------
Amazon Reviews (JSON)
Amazon Metadata (JSON)
Google Trends (CSV)
Common Crawl (WET)

              |
              v

        Apache Spark
   (Distributed Processing)

              |
              v

        Data Processing
   (Schema + Filtering)

              |
              v
```

```
  Parquet Storage (S3)
(reviews / metadata / trends / crawl)

           |
           v

  Read-back Verification
(schema + sample + count)

           |
           v

  Proof-of-Concept Pipeline
```

5 Challenges Encountered

Several challenges were encountered during implementation:

- **Amazon Dataset Schema Issues**
  The raw JSON dataset contains inconsistent and nested structures, causing parsing errors. This was resolved by using a reduced schema and text-based ingestion with `from_json`.
- **AWS Credential Configuration**
  Authentication issues occurred due to incorrect credentials. These were resolved by updating access keys and verifying them using AWS CLI.
- **Incorrect S3 Paths**
  Some file paths were incorrect. This was fixed by inspecting S3 directories and updating configuration files.
- **Large File Processing**
  The Common Crawl dataset is very large. A sample WET file was used to validate the pipeline while maintaining scalability.

6 Next Steps for M3

The next phase will focus on extending the pipeline for analysis:

- Clean and normalize data across all sources
- Expand Amazon dataset to include more fields
- Extract features from Common Crawl text

- Align Google Trends with product categories
- Join multiple datasets
- Generate aggregated insights

7 Data Flow Explanation

The pipeline follows a structured flow:

- Raw data is stored in Amazon S3
- Data is processed using Spark with schema control
- Processed data is written to S3 in Parquet format
- Data is read back for verification

8 Project Structure

The project is organized into modular components:

```
update_CS4265_Project_Jia_Liu/
  config/
    settings.yaml
    env.example.txt
  src/
    ingestion/
    storage/
    preprocessing/
    integration/
    aggregation/
    main.py
  data/  #AWS S3
  docs/
    update_CS4265_JIA_LIU_M1/
    update_CS4265_JIA_LIU_M2/
  requirements.txt
  README.md
```